

Lista BIP-II - Extra

QUESTÃO 1

Escreva o código C em assembly

```
i = 0;

do {
    if(i==3){
        H += 2; // bloco 1
    } else {
        H+=1; // bloco 2
    }
    i++;
} while(i<6);

j = h + 1;

.data
    i : 0
    H : 0
    J : 0

.text
LOOP:
    # if(i == 3)
    LD i
    SUBI 3
    BEQ BLOC01    # Se i == 3, vai pro Bloco 1
    JMP BLOC02    # Se não, vai pro Bloco 2

BLOC01:
    # H += 2
    LD H
```

```

    ADDI 2          # Soma 2 ao valor de H (em vez de sobrescrever com LDI)
    STO H
    JMP CONTINUA   # Pula o Bloco 2 para não executar ambos

BLOC02:
    # H += 1
    LD H
    ADDI 1          # Soma 1 ao valor de H
    STO H

CONTINUA:
    # i++
    LD i
    ADDI 1
    STO i

    # while(i < 6)
    LD i
    SUBI 6
    BEQ FIM        # Se i chegou a 6, sai do loop
    JMP LOOP       # Caso contrário, repete o loop

FIM:
    # j = h + 1
    LD H
    ADDI 1
    STO J
    HLT            # Encerra o programa

```

QUESTÃO 2

- Os endereços das variáveis X, Y e W são 0x6A, 0x6B e 0x6D, respectivamente.
- Os valores iniciais das variáveis X, Y e W são 5, 4 e 0, respectivamente.

a. Descreva o que o código assembly está fazendo

```
LD X
LOOP:
    SUBI 5
    BGE FIM
    LD X
    ADDI 1
    STO X
    JMP LOOP
FIM:
    LD Y
    ADDI 3
    STO W
    HLT
```

Conversão:

```
while(x<5) {
    x += 1
}
w = y+3
```

b. Monte os valores de memória após executar o programa

8 4 2 1

0x6A | x | 5 $\Rightarrow$ 0110 1010

0x6B | y | 4 $\Rightarrow$ 0110 1011

0x6C | w | 0 $\Rightarrow$ 0110 1100

Endereço	Binário	Hexadecimal
00 - LD X	0001 0000 0110 1010	106A
01 - SUBI 5	0011 1000 0000 0101	3805
02 - BGE FIM	0101 1000 0000 0111	5807

03 - LD X	0001 0000 0110 1010	106A
04 - ADDI 1	0010 1000 0000 0001	2801
05 - STO X	0000 1000 0110 1010	086A
06 - JMP LOOP	0111 0000 0000 0001	7001
07 - LD Y	0001 0000 0110 1011	106B
08 - ADDI 3	0010 1000 0000 0011	2803
09 - STO W	0000 1000 0110 1100	086C
10 - HLT	0000 0000 0000 0000	0000

- Quando vamos fazer montagem de programas de forma manual, sempre construímos as instruções de forma sequencial, de modo que quando temos um JMP, apenas referenciamos a primeira instrução de código daquele bloco que foi construído e indentado, mas o processador só irá entender as instruções que lhe forem repassadas de acordo com os locais de memória alocado.